

Crypto Module

Crypto Module - NodeJS

1: `crypto.randomBytes(size)`

Purpose: Generates cryptographically strong random data.

Usage: Useful for creating tokens or unique IDs.

2. `crypto.createHash(algorithm)`

Purpose: Creates a hash for a given input using algorithms like sha256.

Usage: Ensures data integrity (e.g., verifying file changes).

THAPA TECHNICAL - NODEJS YOUTUBE SERIES

crypto.randomBytes(size)

Real-Life Examples:

1. Token Generation for Password Reset

- When users request a password reset, generate a secure random token.
- Example:
- A user clicks "Forgot Password" → A random token is sent via email → The token is verified during the reset process.

1. API Keys and Secret Generation

- Generate secure API keys for third-party integrations.
- Example:
- When creating a developer account on platforms like Twitter or GitHub, a unique API key is generated.

THAPA TECHNICAL - NODEJS YOUTUBE SERIES

crypto.createHash(algorithm)

Real-Life Examples:

1. Password Hashing (Login Systems)

- Hash user passwords before storing them in a database to enhance security.
- Example:
- A user's password is hashed with sha256 and stored. During login, the entered password is hashed and compared.

Common Node.js Built-in Modules

- fs (File System): Handles file operations like reading, writing, and deleting files.
- path: Simplifies working with file and directory paths.
- os (Operating System): Provides information about the operating system and hardware.
- crypto: Offers tools for encryption, hashing, and secure token generation.
- http/https: Allows creation of web servers and handling of HTTP/HTTPS requests.
- events: Supports an event-driven programming model with emitters and listeners.

THAPA TECHNICAL - NODEJS YOUTUBE SERIES

You can also checkout [Node.js official website](https://nodejs.org/en/) to see all built-in modules.